Lab 06 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — "stop/start, my data is still there"

Answer. They are right on one point: `stop` followed by `start` keeps the writable layer, so the written files survive. The data disappears as soon as the **container** does: `podman rm`, `podman run --rm`, a `podman rm -f` followed by a fresh `run` to change a parameter or move to a new image, a `podman-compose down`, or an orchestrator replacing the container.

Why. The writable layer belongs to the container, not to the image or the engine. `stop` destroys nothing: the container still exists (`podman ps -a`), layer included. `rm` deletes it.

Nuance. The layer also survives a reboot of the machine. But "never delete the container" is not a strategy: the first image update forces a new container, hence an empty layer.

Example.

```
podman run --name scribe alpine sh -c 'echo "note" > /data.txt'
podman start -a scribe && podman exec scribe cat /data.txt   # still there
podman rm scribe && podman run --rm alpine ls /data.txt      # No such file or directory
```

Question 2 — The default nginx page

Answer. `html` without `./` is not a path: it is a **named volume** called `html`, created empty on the fly. On its first mount over `/usr/share/nginx/html`, which is not empty in the image, the engine **copies** the image's content into the volume (*copy-up*) — hence the welcome page. Your folder was never mounted. Fix: `-v ./html:/usr/share/nginx/html:ro`.

Why. The `-v` syntax distinguishes bind mount from volume only by the shape of the source: `/`, `./`, `~` → path; anything else → volume name. The volume gets created with no warning.

Nuance. `--mount type=bind,source=html,…` would have failed cleanly (path does not exist) instead of silently creating a volume: that is the main argument for the long syntax.

Example.

```
podman volume ls                        # a volume named "html" has appeared
podman volume inspect html --format '{{.Mountpoint}}'
podman unshare ls $(podman volume inspect html --format '{{.Mountpoint}}')   # index.html,
50x.html: copied from the image
podman rm -f -t 0 web && podman volume rm html
```

Question 3 — Permission denied on the volume

Answer. PostgreSQL runs inside the container as UID 70 (`postgres`). Under rootless, container UID 70 is translated into a UID from your `subuid` range: **100069** on a standard Ubuntu/WSL. The files therefore belong to 100069, and your user 1000 has no rights on them — which is the point. `podman unshare ls -ln …` lists them by entering Podman's user namespace.

Why. The user namespace maps only your UID onto root; every other container UID points into the `100000-165535` range. The kernel enforces permissions on the **real** UIDs.

Nuance. The parent directory (`volumes/pgdata`) does belong to 1000: only what the container writes carries a translated UID. `sudo ls` would work too, but you would be using root to bypass an isolation you asked for.

Example.

```
podman unshare cat /proc/self/uid_map
#          0       1000          1
#          1     100000      65536
podman unshare ls -ln ~/.local/share/containers/storage/volumes/pgdata/_data | head -3   # 70
70
ls -ln ~/.local/share/containers/storage/volumes/                                        #
1000 1000 pgdata
```

Question 4 — alpha or bravo?

Answer. **`alpha`**. The second container finds an already-initialised `PGDATA` in the volume and does not rerun `initdb`: `POSTGRES_PASSWORD=bravo` is ignored, and the password stored in the database stays `alpha`.

Why. The official image's entrypoint script creates the cluster and the user only when the directory is empty. The `POSTGRES_*` variables serve that initialisation and nothing else.

Nuance. To change the password, go through SQL (`ALTER USER postgres PASSWORD 'bravo'`), or delete the volume — data included. This trap resurfaces in CI when a volume lingers between two runs.

Example.

```
podman logs db 2>&1 | grep -c initdb     # 0 on the second start: nothing initialised
podman exec db psql -U postgres -c "ALTER USER postgres PASSWORD 'bravo';"
```

Question 5 — Which kind for which need

Answer. (a) **Bind mount**: you edit the sources on the host and the container sees them without a rebuild. (b) **Named volume**: engine-managed data, correct permissions, backed up with `volume export`. (c) **Bind mount with `:ro`**: a file versioned with the project that the container must not modify. (d) **tmpfs** if RAM allows it, otherwise a dedicated bind mount or volume — never the writable layer, which is slow and bloats the container.

Why. The criterion is: who has to see and manage the files? You → bind mount; the engine → volume; nobody after shutdown → tmpfs.

Nuance. A 2 GB tmpfs consumes 2 GB of RAM, counted against `--memory`. For (d), a named volume that the container cleans up itself is often more realistic.

Example.

```
podman run --rm --tmpfs /tmp/pdf:size=2g alpine df -h /tmp/pdf
```

Question 6 — 1000 cannot write into 1000's folder

Answer. The two "1000"s are not the same user. The folder belongs to **host** UID 1000, which the container sees as **root** (`0`). UID 1000 **inside the container** is, seen from the host, 100999. To the kernel, that is a stranger trying to write into your folder. Fixes: (1) `:U` on the mount, so the engine `chown`s the folder to the container's UID; (2) `--userns=keep-id`, so your UID keeps the number 1000 inside the container and the file really is yours.

Why. `uid_map`: `0 1000 1` and `1 100000 65536`. Container UID 1000 falls on the second line: `100000 + (1000 - 1) = 100999`.

Nuance. `:U` changes the owner **on the host**, permanently (question 7). `keep-id` is meant for development tools; it does not suit images whose process must run under a fixed UID (postgres = 70).

Example.

```
podman run --rm --user 1000 -v ./output:/out:U alpine sh -c 'echo x > /out/f'     # OK
podman run --rm --userns=keep-id -v ./output:/out alpine sh -c 'id -u; echo y > /out/g'   # 1000, OK
```

Question 7 — After `:U`

Answer. On the host, `ls -ln output` shows the folder and the files owned by UID **100999** (container UID 1000, translated), and you can no longer write there without Podman. To take it back: `podman unshare chown -R 0:0 output` — "root" inside the namespace is you.

Why. `:U` is a real recursive `chown`, performed by the engine inside the user namespace. Removing the container does not undo it.

Nuance. In the sandbox these labs were written in, the subuid range starts at 1001: the file shows up there with UID 1001. On your WSL it is 100999. The mechanism is the same; only the range differs (`cat /etc/subuid`).

Example.

```
ls -ln output            # -rw-r--r-- 1 100999 1000 … f
podman unshare chown -R 0:0 output && ls -ln output    # 1000 1000
```

Question 8 — Fourteen anonymous volumes

Answer. They come from images that declare `VOLUME` (postgres, mysql, redis, jenkins…) started **without** `-v`: every `run` creates an anonymous volume, and `podman rm` does not remove it. For each one, `podman ps -a --filter volume=<name>` tells you which container uses it (none → orphan), and `podman volume inspect` gives its creation date. To stop the growth: name your volumes, and remove containers with `rm -v`. `podman volume prune` deletes **every** volume not used by an existing container — including a named `pgdata` whose container was recreated yesterday.

Why. The engine cannot tell "temporary" from "precious"; it only knows "mounted by an existing container". A named volume whose container was removed is, to `prune`, garbage.

Nuance. `podman volume ls --filter dangling=true` lists the volumes no existing container mounts, without deleting anything — but a named `pgdata` whose container was just removed

shows up there too. That list is for **reading**, not for piping straight into `rm`.

Example.

```
podman image inspect postgres:16-alpine --format '{{.Config.Volumes}}'   # map[/var/lib/postgresql/data:{}]
podman volume ls --filter dangling=true
podman ps -a --filter volume=pgdata --format '{{.Names}}'
```

Question 9 — `tar` on a running database

Answer. While PostgreSQL writes, the data files and the journal (WAL) change constantly; `tar` reads them one after the other, at different moments. The archive is a mix of states: at best PostgreSQL replays the WAL and recovers, at worst it refuses to start (`invalid checkpoint record`). The two correct ways: a **logical backup** (`pg_dump`, consistent while running, portable), or a **physical backup with the container stopped** (`volume export`, `tar`). For a daily backup: `pg_dump`, which needs no downtime.

Why. `pg_dump` opens a transaction and reads a consistent snapshot of the database. A file copy has no such notion.

Nuance. A backup that was never restored does not exist: a restore test into a throwaway container is part of the procedure. Real tools for online physical backups (`pg_basebackup`, pgBackRest) exist, but they are beyond this lab.

Example.

```
podman exec db pg_dump -U postgres > dump.sql          # while running, consistent
podman stop db && podman volume export pgdata -o pgdata.tar && podman start db   # physical, stopped
```

Question 10 — `volume export` versus `tar` in a container

Answer. The second command starts a throwaway Alpine container with **two** mounts: the `pgdata` volume on `/data`, read-only, and your current directory on `/backup`; `tar` archives `/data` into `/backup/pgdata.tgz`, which therefore appears in your folder. It needs a container because, seen from the host, the volume's files belong to UID 100069 and your `tar` cannot read them — inside the container, they are readable. This method **also works with Docker**; `podman volume export` is Podman-only.

Why. A utility container is the universal way to "see a volume with the right permissions". `podman volume export` does the same thing internally, without a container.

Nuance. `podman unshare tar czf … $(podman volume inspect pgdata --format '{{.Mountpoint}}')` is a third route, Podman-only. In every case: container stopped.

Example.

```
podman run --rm -v pgdata:/data:ro -v ./:/backup alpine tar czf /backup/pgdata.tgz -C /data .
tar tzf pgdata.tgz | head -3
```

Question 11 — A secret in tmpfs

Answer. The writable layer sits **on the host's disk** (`~/.local/share/containers/storage/overlay/…`): a secret leaves a trace there for as long as the container exists, and possibly in a `podman commit`, a `podman export`, or a disk snapshot. A tmpfs is **in memory**: never written to disk, wiped at shutdown, unreachable through the engine's storage. The price: its size comes out of RAM and counts against `--memory`.

Why. tmpfs is a kernel filesystem backed by memory (and by swap, if enabled — which is why bounding `size=` matters).

Nuance. A tmpfs does not protect against a `podman exec` while the container runs: it protects against **persistence**, not against live access.

Example.

```
podman run --rm --tmpfs /run/secrets:size=1m alpine sh -c 'echo token > /run/secrets/t; df -h
/run/secrets'
```

Question 12 — Permission denied on RHEL

Answer. RHEL runs **SELinux** in *enforcing* mode: the files in `./conf` carry your home directory's label (`user_home_t`), which containerised processes are not allowed to read, root or not. Fix: `-v ./conf:/etc/app:ro,Z`, which relabels the directory as `container_file_t`. On Ubuntu/WSL there is no SELinux (AppArmor instead); the option is accepted and ignored. Writing it costs nothing and makes the command portable.

Why. SELinux adds **mandatory** access control, independent of UIDs and `chmod`. `--user 0` changes nothing: the decision is made on labels.

Nuance. `:Z` labels the directory for **this** container; `:z` for all of them. Never apply `:Z` to a system directory (`/home`, `/etc`): relabelling breaks access for the rest of the system.

Example.

```
podman run --rm -v ./conf:/etc/app:ro,Z alpine cat /etc/app/app.yaml   # RHEL: OK; WSL: OK too
ls -Z conf   # RHEL: …container_file_t…; WSL: ? (no label)
```

Question 13 — Two APIs, one `uploads` volume

Answer. Each container mounts **its own** local volume: a document uploaded to replica 1 is invisible to replica 2, and downloads fail every other time. If the two run on different machines, the volume cannot even be shared. Companies put user files in **object storage** (S3, MinIO, Azure Blob), reachable over HTTP from any replica; the application keeps **no local state**. That is the condition for a container to be disposable: you can kill one, start three, move them — nothing is lost.

Why. A volume is local to one host and one engine. Shared state belongs in a service designed for it (database, object storage, network cache).

Nuance. With a single replica on a single machine, the volume works — which is exactly what keeps the mistake invisible until the day you scale. Kubernetes has shared volumes (`ReadWriteMany`, NFS), but they remain the exception.

Example.

```
podman run -d --name api1 -v uploads1:/app/uploads … ; podman run -d --name api2 -v
uploads2:/app/uploads …
podman exec api1 touch /app/uploads/doc.pdf ; podman exec api2 ls /app/uploads    # empty
```